

Key Takeaways

1. **Always inherit from `nn.Module`** for custom models.
2. **Call `super().__init__()`** in your model's constructor.
3. **Use `nn.Sequential`** for simple feed-forward networks.
4. **Never call `.forward()` directly** – call the model instance.
5. **Loss functions** are callable objects; ensure input shapes match.
6. **Optimizer** requires `model.parameters()` and learning rate.
7. **Training loop pattern:** `zero_grad` → `backward` → `step`.

These notes cover everything Nitish explained. Write them in your copy with headings and code blocks. Happy learning!

Notes on PyTorch Dataset & DataLoader Classes

1. Problem with Batch Gradient Descent

In batch gradient descent, the entire dataset is used at once to compute gradients and update parameters.

Problems:

- **Memory inefficient:** Entire dataset must fit in RAM (e.g., 10GB of images cannot be loaded at once).
- **Slow convergence:** Parameters update only once per epoch; updates are infrequent and slow.

Solution: Mini-batch gradient descent – split data into smaller batches, update parameters after each batch.

2. Mini-Batch Gradient Descent

- Divide dataset into batches (e.g., 1000 rows → 10 batches of 100 rows each).
- For each batch: forward pass → loss → backward pass → update parameters.
- Repeat for all batches → one epoch completed.
- Faster convergence and memory efficient.

3. Problems with Manual Mini-Batch Implementation

Manually creating batches using loops has several issues:

Problem	Description
No standard interface	Data loading logic (e.g., reading images from folders) has no dedicated place.
No easy transformations	Cannot apply resizing, normalization, etc., before

Problem	Description
	batching.
Shuffling & sampling	No built-in way to shuffle data or create balanced batches.
Batch management & parallelization	No support for parallel batch creation.

4. Dataset & DataLoader – The Solution

PyTorch provides two classes to decouple **data loading** from **training loop**:

- **Dataset:** Knows where data is stored and how to load individual samples.
- **DataLoader:** Creates batches, handles shuffling, parallelization, and collation.

Workflow Diagram:

Data Source → Dataset (loads rows one by one) → DataLoader (creates batches) → Training Loop

5. Creating a Custom Dataset Class

You must inherit from `torch.utils.data.Dataset` and implement three methods:

```
from torch.utils.data import Dataset

class CustomDataset(Dataset):
    def __init__(self, features, labels):
        # Constructor: load data from source (CSV, folders, etc.)
        self.features = features
        self.labels = labels

    def __len__(self):
        # Return total number of samples
        return len(self.features)

    def __getitem__(self, index):
        # Return the sample (features, label) at given index
        # Apply transformations here if needed
        x = self.features[index]
        y = self.labels[index]
        return x, y
```

Explanation:

- `__init__`: Load data into memory (e.g., read CSV, list image paths).
- `__len__`: Used by DataLoader to know number of batches.
- `__getitem__`: Fetches one sample by index; ideal place for transformations.

6. Using DataLoader

DataLoader wraps a Dataset and provides batching, shuffling, parallel loading.

```
from torch.utils.data import DataLoader

dataloader = DataLoader(
    dataset=custom_dataset,    # your Dataset object
    batch_size=32,            # samples per batch
    shuffle=True,              # shuffle data every epoch
    num_workers=2,             # parallel processes for loading
    pin_memory=True,           # faster GPU transfer
    drop_last=False,           # drop incomplete last batch
    collate_fn=None            # custom collation function
)
```

Key Parameters:

- `batch_size`: Number of samples per batch.
- `shuffle`: Randomize order each epoch (use `False` for time series).
- `num_workers`: Number of subprocesses for parallel loading.
- `drop_last`: If `True`, discard the last incomplete batch.
- `collate_fn`: Function to combine samples into a batch (custom padding, etc.).

7. How DataLoader Works Internally

1. **Sampler** generates indices (random or sequential).
2. Indices are grouped into chunks of size `batch_size`.
3. For each chunk, DataLoader calls `__getitem__` on Dataset to fetch samples.
4. **Collate function** combines samples into a batch tensor.
5. Batches are yielded to training loop.

Sampler Types:

- `RandomSampler` (`shuffle=True`) – shuffles indices.
- `SequentialSampler` (`shuffle=False`) – indices in order.
- Custom sampler for imbalanced datasets (e.g., ensure minority class appears in every batch).

Collate Function Use Case: When samples have different sizes (e.g., sentences of different lengths), custom collate function can pad them to equal size.

8. Applying Transformations

Transformations are applied inside `__getitem__` before returning the sample.

```
def __getitem__(self, index):
    x = self.features[index]
```

```

y = self.labels[index]
# Apply transformations here
x = self.transform(x)    # e.g., resize, normalize, to tensor
return x, y

```

Common transformations for images: resize, random crop, color jitter, convert to tensor.

9. Parallelization with num_workers

- `num_workers=0` (default) → all batching done in main process (sequential).
- `num_workers>0` → multiple subprocesses load and collate batches in parallel.
- Faster batch preparation, especially for large datasets (images, text).

10. Complete Example: Synthetic Dataset

```

import torch
from torch.utils.data import Dataset, DataLoader
from sklearn.datasets import make_classification

# Create synthetic data
X, y = make_classification(n_samples=10, n_features=2, n_classes=2)
X = torch.tensor(X, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32)

# Define custom Dataset
class MyDataset(Dataset):
    def __init__(self, features, labels):
        self.features = features
        self.labels = labels
    def __len__(self):
        return len(self.features)
    def __getitem__(self, idx):
        return self.features[idx], self.labels[idx]

# Create Dataset and DataLoader
dataset = MyDataset(X, y)
dataloader = DataLoader(dataset, batch_size=2, shuffle=True)

# Iterate over batches
for batch_idx, (batch_features, batch_labels) in enumerate(dataloader):
    print(f"Batch {batch_idx+1}:")
    print(f"Features: {batch_features}")
    print(f"Labels: {batch_labels}")
    print("---")

```

Output (example – actual values vary due to shuffle):

```

Batch 1:
Features: tensor([[ -0.1234,  1.2345],
                  [ 0.5678, -0.9876]])
Labels: tensor([0., 1.])
---
Batch 2:
Features: tensor([[ 1.1234, -0.3456],
                  [-0.7890,  0.4567]])
Labels: tensor([1., 0.])
---
...

```

11. Integration into Breast Cancer Training Pipeline

Before (Batch Gradient Descent):

- Entire `X_train`, `y_train` fed to model at once.
- One forward/backward per epoch.

After (Mini-batch with Dataset/DataLoader):

```

# Create custom Dataset
class BreastCancerDataset(Dataset):
    def __init__(self, features, labels):
        self.features = features
        self.labels = labels
    def __len__(self):
        return len(self.features)
    def __getitem__(self, idx):
        return self.features[idx], self.labels[idx]

# Create train/test datasets
train_dataset = BreastCancerDataset(X_train_tensor, y_train_tensor)
test_dataset = BreastCancerDataset(X_test_tensor, y_test_tensor)

# Create DataLoaders
train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=32, shuffle=False)

# Training loop
model = YourModel()
optimizer = torch.optim.SGD(model.parameters(), lr=0.01)
criterion = torch.nn.BCELoss()

for epoch in range(num_epochs):
    for batch_features, batch_labels in train_loader:
        # Forward pass
        outputs = model(batch_features)
        loss = criterion(outputs, batch_labels)

```

```
# Backward pass & update
optimizer.zero_grad()
loss.backward()
optimizer.step()
```

12. Summary of Benefits

Manual Approach Problem	Dataset + DataLoader Solution
No standard interface	Dataset class with <code>__getitem__</code>
No transformation place	Apply transforms in <code>__getitem__</code>
No shuffling/sampling	<code>shuffle=True</code> and custom samplers
No parallelization	<code>num_workers</code> parameter

Key Takeaway: Always use `Dataset` and `DataLoader` for any real-world PyTorch project. They are essential for efficient, scalable, and clean code.

End of Notes

PyTorch - Building an Artificial Neural Network (ANN) on Fashion-MNIST

Notes for Your Copy

1. Recap of Previous PyTorch Concepts (6 Videos)

Before building the ANN, we covered these fundamentals:

Video	Topic	Key Idea
1	Introduction to PyTorch	Basics, tensors, why PyTorch
2	Tensors	Creating and manipulating tensors
3	Autograd	Automatic differentiation
4	Training Pipeline	Manual training loop
5	<code>nn.Module</code>	Organizing neural networks
6	Dataset & DataLoader	Batching and loading data